Language Theory and Automata

2nd Year Applied Artifical Intelligence -2





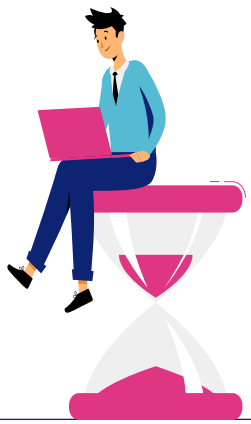
Dr-Ing Ines Baccouche ©Pristini 2024-2025

Course Outlines

Chapter 1 1
Introduction to langage theory

Chapter2 2
Preliminary notions of regular expressions

Chapter3 3
Automata Principle



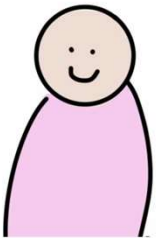
Chapter4 4
Determisit and non-determinist Automata : DFA, NFA,

Chapter5 5
Application of Automata : Turing Machine

Chapter6 6
Application of Automata in IA

Whyyy !!??





Link of video : https://www.youtube.com/watch?v=SV57Yv8BXBc

Chapter 1

Introduction to computation theory



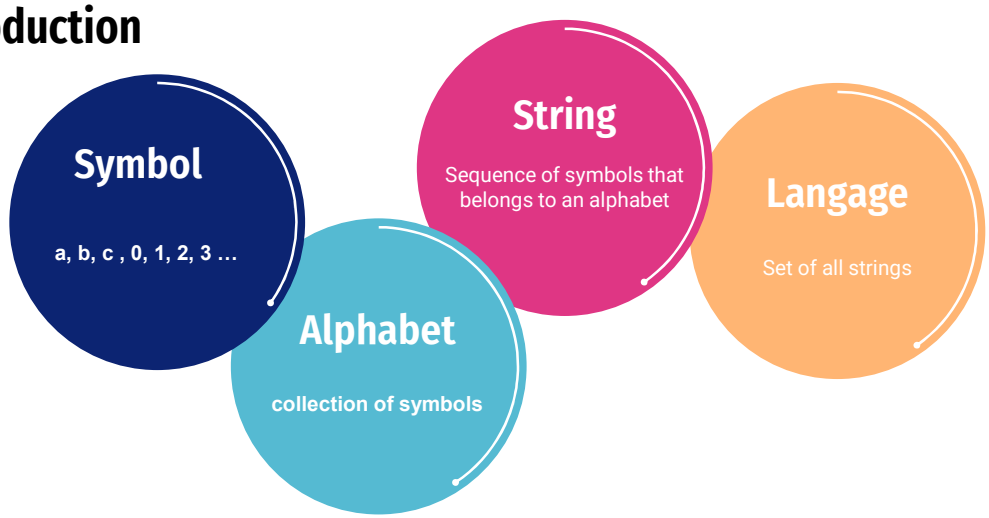
Introduction

- The purpose of this course is an introduction to the theory of formal languages.
- Generally speaking, languages are the natural means of communication. They allow humans to exchange information and ideas, they also allow them to communicate with machines.
- The languages used in everyday life between human beings are said to be natural. They are usually informal and ambiguous and require all the subtlety of a human brain to be interpreted correctly.
- Human-made languages to communicate with computers are artificial languages. They must be formalized and unambiguous in order to be interpreted by a machine.

Introduction

- The theory of languages and automata has a particular aspect in learning computer science.
- It is a subject which belongs to the fundamental branch of computer science, as the concepts covered are found in many computer science disciplines, for example:
  - ✓ **Design of processors,**
  - ✓ **Compilation of computer programs,**
  - ✓ **Automatic translation of natural languages,**
  - ✓ **Artificial intelligence,**
  - ✓ **Bioinformatics,**
  - ✓ **Verification of critical embedded programs,**
  - ✓ **Cybersecurity**
- The theory of languages and automata is a teaching essential in any computer science curricular

Introduction



Symbol: a
Alphabet: {a, b}
String: aabb
Language: {ε, ab, aabb, aaabbb, …}

Introduction

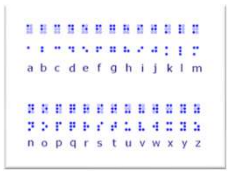
Alphabet

- **Definition :** An alphabet is a finite set of elements called letters or symbols.

  An alphabet will usually be designated by a capital Greek letter.

  Σ = {a, b, c}, **Γ** = {♥, ♦, ♣, ♠}, **Δ** = {0, 1}, **Φ** = {→,←, ↑, ↓} are alphabets.

- **Examples**
  - ✓ X= {0,1} : binary alphabet
  - ✓ A= {a, b, c…,A,B….é,ç….} : English langage alphabet
  - ✓ B= {int, if, else, printf, scanf……} : C Programming langage
  - ✓ The biologist who study DNA will use a four-letter alphabet **{A, C, G, T}** for the four constituents of the genes: Adenine, Cytosine, Guanine and Thymine.



Braille Alphabet



Introduction

String

- **A string** (or sometimes a word) is a finite sequence of symbols chosen from some alphabet

  Example: 01101 and 111 are strings from the binary alphabet Σ = {0, 1}

- **Empty string**: the string with zero occurrences of symbols This string is denoted by ε and may be chosen from any alphabet whatsoever.

- **Length of a string**: the number of positions for symbols in the string
  Example: 01101 has length 5
  • There are only two symbols (0 and 1) in the string 01101, but 5 positions for symbols

- **Notation of length** of w: |w| Example: |011| = 3 and | ε | = 0

Introduction

String

- A string is a finite sequence of symbols chosen from an alphabet

Alphabets	Strings
{0,1}	0, 1, 1101,001010
{a, b}	aab, bba, ababa
{0, 1, 2, 3, 4, 5, 6, 7, 8, 9}	22,109,16789,41
{ they, guitar, play, I , with}	they play guitar, I play
{if, then, else, C, X,=,0}	if C then X=0, else C

Introduction

String

Powers of an alphabet

If Σ is an alphabet, we can express the set of all strings of a certain length from that alphabet by using the exponential notation:

- Σ^k: the set of strings of length **k**, each of whose is in Σ
- **Examples:**
  - → Σ^0 : {ε }, regardless of what alphabet Σ is. That is ε is the only string of length 0
  - → If Σ = { 0, 1 }, then:
    1. Σ^1 = { 0, 1 }
    2. Σ^2 = {00, 01, 10, 11 }
    3. Σ^3 = {000, 001, 010, 011, 100, 101, 110, 111 }

Note: confusion between Σ and Σ^1 :
- Σ is an alphabet; its members 0 and 1 are symbols
- Σ^1 is a set of strings; its members are strings (each one of length 1)

Introduction

String

Kleen Star

Stephen Kleene



- Σ^*: The set of all strings over an alphabet Σ
  - → **{0, 1}*** = {ε, 0, 1, 00, 01, 10, 11, 000, . . .}
  - → $\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \ldots \cup \Sigma^n$
- The symbol * is called **Kleene star** and is named after the American mathematician and logician Stephen Cole Kleene.
- $\Sigma^+ = \Sigma^1 \cup \Sigma^2 \cup \ldots$ Thus: $\Sigma^* = \Sigma^+ \cup \{\varepsilon\}$ ('+' for all string expect ε)

Introduction

String

Concatenation

- Define the binary operation '.' called concatenation on Σ^* as follows:
- If $a_1a_2a_3 \ldots a_n$ and $b_1b_2 \ldots b_m$ are in Σ^*, then

  $a_1a_2a_3 \ldots a_n . b_1b_2 \ldots b_m = a_1a_2a_3 \ldots a_nb_1b_2 \ldots b_m$

- Thus, strings can be concatenated yielding another string:
  If x are y be strings then $x.y$ denotes the concatenation of x and y, that is, the string formed by making a copy of x and following it by a copy of y
- **Examples:**
  → x = 01101 and y = 110 Then xy = 01101**110** and yx = **110**01101
  → For any string ω, the equations εω = ωε = ω
  That is, ε is the identity for concatenation
  (when concatenated with any string it yields the other string as a result)

Introduction

String

Exercise: Let the alphabet A = {a, b}.

1. Given the words **u = aa** and **v = bab**, write the words uv, $(uv)^2$ and u^3v.
2. State all the words of length 2 defined on A.
3. Let the sets E1 = {u.v/u ∈ A+, v ∈ A+} E2 = {u.v/u ∈ A+, v ∈ A∗} E3 = {u.v/u ∈ A∗ , v ∈ A∗}
   What do these sets correspond to?

Solution :

1. uv = aabab, (uv) = aababaabab and u^3v = aaaaaabab.
2. Words of Length 2 = {aa, ab, ba, bb}
3. E1 = {u ∈ A∗/|u| ≥ 2} = set of words with at least 2 symbols,
   E2 = A+ ,
   E3 = A∗

Introduction

Language

- If Σ is an alphabet, and L ⊆ Σ^*, then L is a (formal) language over Σ.
- **Language**: A (possibly infinite) set of strings all of which are chosen from some Σ^*
- **Examples**:

→ Programming language C Legal programs are a subset of the possible strings that can be formed from the alphabet of the language (a subset of ASCII characters)

→ English or French ….

Introduction

Language

- A language can be finite or infinite.

Example: Let Σ={a,b}

- **L1 = {aa, abba, bba } is a finite language**
- **L2= {ab, aaaab, a.....b} is an infinite language**

Introduction

Language

Examples

- The language of all strings consisting of n 0s followed by n 1s (n ≥ 0):
  {ε, 01, 0011, 000111, . . . }
- The set of strings of 0s and 1s with an equal number of each:
  {ε, 01, 10, 0011, 0101, 1001, . . . }
- Σ^* is a language for any alphabet Σ
- ∅, the empty language, is a language over any alphabet
- {ε}, the language consisting of only the empty string, is also a language over any alphabet
  NOTE: ∅ ≠{ε } since ∅ has no strings and {ε } has one
- { w | w consists of an equal number of 0 and 1 }
- $\{ 0^n 1^n \mid n \geq 1 \}$
- $\{ 0^i 1^j \mid 0 \leq i \leq j \}$

17 TLA- Dr-Ing Ines Baccouche - Bachelor GLSI-2 2024 PRISTINI SCHOOL OF A

Introduction

Language

Operations on Languages

Union	$L_1 \cup L_2$	$\{x \mid x \in L_1 \text{ ou } x \in L_2\}$
Intersection	$L_1 \cap L_2$	$\{x \mid x \in L_1 \text{ et } x \in L_2\}$
Difference	$L_1 - L_2$	$\{x \mid x \in L_1 \text{ et } x \notin L_2\}$
Complement	$\bar{L}$	$\{x \in \Sigma^* \mid x \notin L\}$
Concatenation	$L_1 L_2$	$\{xy \mid x \in L_1 \text{ et } y \in L_2\}$
Auto-concatenation	$\overbrace{L \ldots L}^{n}$	L^n
Closure(Kleene star)	L^*	$\bigcup_{k \geq 0} L^k$

18 TLA- Dr-Ing Ines Baccouche - Bachelor GLSI-2 2024 PRISTINI SCHOOL OF A

Introduction

Language

Operations on Languages

- The **union** of two languages L and M, denoted **L ∪ M**, is the set of strings that are in either L, or M, or both.

Example
If **L** = {001, 10, 111} and **M** = {ε, 001} then **L ∪ M** = {ε, 001, 10, 111}

Symbol: '|' (pipe)
Combines two languages into a new language that contains all strings from both languages.

Introduction

Language

Operations on Languages

- The **concatenation** of languages L and M, denoted L.M or just LM , is the set of strings that can be formed by taking any string in L and concatenating it with any string in M.

Example
If L = {001, 10, 111} and M = {ε, 001} then L.M = {001, 10, 111, 001001, 10001, 111001}

Symbol: '.' (dot) or implicit juxtaposition
Concatenates two languages, creating a new language containing all possible combinations of strings where one string is from the first language, and the other is from the second.

Example: any string that starts with 'a' followed by 'b': ab

Example: To match either 'apple' or 'banana': apple|banana

Introduction

Language

Operations on Languages

- The **closure** of a language L is denoted L^* and represents the set of those strings that can be formed by taking any number of strings from L, possibly with repetitions (i.e., the same string may be selected more than once) and concatenating all of them.

Examples:

- If L = {0, 1} then L^* is all strings of 0 and 1
- If L = {0, 11} then L^* consists of strings of 0 and 1 such that the 1 come in pairs, e.g., 011, 11110 and ε. But not 01011 or 101

Symbol: **'*'** (asterisk)
Creates a new language containing all possible finite concatenations (including the empty string) of strings from the original language.

Example: To match zero or more 'a's: a* matches '', 'a', 'aa', 'aaa', ...

Introduction

Language

Operations on Languages

- **Kleene plus**

Symbol: **'+'** (plus)

Example: one or more 'b's: b+ matches 'b', 'bb', 'bbb', ...

- **Positive Closure (Positive Plus):**

Symbol: + (plus) with a positive sign
Example : at least one digit: **\d+** matches '1', '123', '9999', ...

Introduction

Language

Operations on Languages

Operation	Description	Symbol	Example (Alphabet {a, b})
Concatenation	Combines two languages to form a new language of concatenated strings.	. (dot)	**ab** (strings starting with 'a' followed by 'b')
Union (Alternation)	Creates a new language containing strings from either of the languages.	| (pipe)	**(a|b)** (string **a** or string **b**)
Kleene Star (Closure)	Generates a language with all possible concatenations (including none) of strings from the original language.	* (asterisk)	**a*** (zero or more 'a's)
Kleene Plus	Similar to Kleene Star, but requires at least one concatenation.	+ (plus)	**b+** (one or more 'b's)
Optional (Zero or One)	Specifies that the preceding element is optional (appears zero or one time).	? (question mark)	**ab?** (string 'a' followed by **optional** 'b')
Positive Closure	Specifies that the preceding element must appear at least once.	+ (plus) positive sign	**ab+** (string 'a' followed by one or more 'b's)
Complement (Negation)	Represents all strings not in a given language.	¬ (negation symbol)	**^a** (strings **not starting** with 'a')
Intersection	Represents the set of strings that are common to two languages.	∩ (intersection symbol)	**{a, b}** ∩ **{b}** (strings with 'b')
Difference	Represents all strings in the first language but not in the second.	\ (backslash)	**{a, b}** \ **{b}** (strings with 'a'
Grouping	Groups elements to define the order of evaluation in the regex.	( ) (parentheses)	**(ab)*** (zero or more 'ab' sequences)

23

Preliminary notions

Regular Expression

- A regular expression (regex) is a string of characters defining a **pattern** to be found in a block of text.
- A regex can be constructed to allow multiple possible matches, while restricting other possibilities

Basis:

The basis consists of three parts:

- The constants ε and ∅ are regular expressions, denoting the language {ε } and ∅, respectively. That is L(ε) = {ε} and L(∅) = ∅.
- If **a** is a symbol, then **a** is a regular expression. This expression denotes the language { a }, i.e., L(a) = { a }.

24 TLA- Dr-Ing Ines Baccouche - Bachelor GLSI-2 2024

Preliminary notions

Regular Expression

- A **regular expression** is a declarative way to describe the construction of **strings** in a language L.
- A language is said to be regular if and only if it is denoted by a regular expression. We denote the language generated by a regular expression E as L(E)

➔ **A regular expression, therefore, describes a set of possible strings according to a given syntax**

Preliminary notions

Regular Expression

Examples

(a|b)*
This regular expression matches zero or more occurrences of either 'a' or 'b'. It effectively matches any combination of 'a' and 'b', including empty strings.

{ ‘’, ‘a’, ‘b’, 'ab’, 'ba’, 'aab’, 'bbababab’…}

(ab|a)

The result is whether ‘ab’ or ‘a’ :

{ ‘a’, ‘ab’ }

Preliminary notions

Regular Expression

❑ Application

Let R and S be two regular expressions defined as follows:
R = a(a|b)ba
S = (ab)* | (ba)*

1. Find a string included in the language denoted by R that is not included in the language denoted by S.
2. Find a string included in the language denoted by S that is not included in the language denoted by R.
3. Find a string included in both the language denoted by R and the language denoted by S.
4. Find a string that is included in neither the language denoted by R nor the language denoted by S.

Preliminary notions

Regular Expression

❑ Exercice1

Give the elements that corresponds to there RegEx

- **ab*|c =**
- **(a+ b) =**
- **b*ab*ab***
- **(a|b*)* (c|d)***

Preliminary notions

Regular Expression

❑ **Exercice1**

Give the Regex on {a,b,c} alphabet that :

- Matches all strings
- Matches exactely words with 3 a,
- Matches words with at least 3 a

Is these equalities true or not ?

- (a* | b*) = (a | b)* ?
- (a | b)* b = (a | b)* ?
- (a*b)* = (a | b)*b ?

Preliminary notions

Regular Expression

❑ **Exercise2**

Provide the regular expressions for the following languages constructed over the alphabet {a, b, c}:

1. All strings :
2. Three-letter strings
3. All strings without 'b'.
4. All strings containing at **most** one occurrence of the letter 'a'.
5. All strings containing at **least** one occurrence of the letter 'a'.
6. All strings in which each 'a' is followed by a 'b'.
7. All strings of even length
8. All strings with the prefix 'ab'.
9. All strings with the suffix 'ab'.

Preliminary notions

Regular Expression

❑ **Exercise2**

Provide the regular expressions for the following languages constructed over the alphabet {a, b, c}:

1. All strings : (a|b|c)* = (a* b* c*)*
2. Three-letter strings (a|b|c){3}
3. All strings without 'b'. (a|c)*
4. All strings containing at **most** one occurrence of the letter 'a'. (b|c)*|(b|c)*a(b|c)*
5. All strings containing at **least** one occurrence of the letter 'a'. (a|b|c)* a (a|b|c)* =
6. All strings in which each 'a' is followed by a 'b'. (b |c |ab)*
7. All strings of even length. (..) *
8. All strings with the prefix 'ab'. ab(a|b|c)*
9. All strings with the suffix 'ab'. (a|b|c)*ab

Preliminary notions

Regular Expression

❑ **Exercise3**

Let Σ be the alphabet ∑ = {a, b}.

1. L1 = {w | w begins with a}
2. L2 = {even-length strings}

➔ Propose regular expressions for these two languages.

Preliminary notions

Regular Expression

Notation

Token	Matches	Set Equivalent
`\d`	Any digit	`[0-9]`
`\s`	Any whitespace character (space, tab, newlines)	`[ \t\n\r\f]`
`\w`	Any 'word character' - letters, numbers, and underscore	`[A-Za-z0-9_]`
`\D`	The inverse of `\d` i.e. any character except a digit	`[^0-9]`
`\S`	Any non-whitespace character	`[^ \t\n\r\f]`
`\W`	Any non-word character	`[^A-Za-z0-9_]`

Preliminary notions

Regular Expression

The use of regular expressions: examples of applications

- **Definition of lexical analysers (compilers).**
- Used in operation systems like UNIX (a Unix-style):
  [A-Z] [a-z] * [ ] [A-Z] [A-Z] : represents capitalised strings followed by a space and two capital letters.

➔ This expressions represents patterns in text that could be a city and a state,

Preliminary notions

Regular Expression

❑ **Exercise4**

- Give this Regex Over the alphabet {0,1,2,3,4,5,6,7,8,9} , d=(0-9)

→ Regular expression that denotes numbers of only three digits : \d{3}

→ Regular expression that denotes numbers that could be divided by 5 : (0-9)*(0|5)

→ Regular expression that denotes numbers that starts with 4 and ends with 7: 4.*7

Preliminary notions

Regular Expression

❑ **Exercise5**

- Create a regular expression for all strings that represent valid email addresses.
- An email address should consist of the following:
  - ✓ A series of one or more alphanumeric characters (letters and digits).
  - ✓ Followed by the "@" symbol.
  - ✓ Followed by a series of one or more alphanumeric characters (letters and digits).
  - ✓ Followed by a dot (".") symbol. Ending with a top-level domain (TLD) such as "com," "org," "net," etc..

[a-zA-Z0-9]+@[a-zA-Z0-9]+\.[a-zA-Z]{2,}